

- particular array index. an index will be generated i.e every key is stored in a
- Using the generated index, access the data located in that array index.
  - In case of absence of data, create one and insert the data item (key and value) into it and increment the size of hash table.
  - In case the data exists, probe through the subsequent elements (looping back if necessary) for free space to insert new data item.
    - This probing will continue until we reach the same element again (from where we began probing)
    - Here, unlike Linear Probing, probing will be done according to the following formula  
 $(\text{currentPosition} + h) \% \text{arraySize} \Rightarrow \text{Linear Probing}$   
 $(\text{currentPosition} + (h * h)) \% \text{arraySize} \Rightarrow \text{Quadratic Probing}$   
 where  $h = 1, 2, 3, 4$  and so on.
  - To display all the elements of hash table, element at each index is accessed (via for loop).
  - To remove a key from hash table, we will first calculate its index and delete it if key matches, else probe through elements until we find key or an empty space where not a single data has been entered (means data does not exist in the hash table).
  - Exit

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define TABLE_SIZE 10
int h[TABLE_SIZE] = {NULL};
```

```
void insert() {
    int key, index, i, flag = 0, hkey;
    printf("Enter a value to insert\n");
    printf("into hash table\n");
    scanf("%d", &key);
```

```

hkey = key % TABLE_SIZE;
for (i = 0; i < TABLE_SIZE; i++) {
    index = (hkey + i * i) % TABLE_SIZE;
    if (h[index] == NULL) {
        h[index] = key; break; }
}
if (i == TABLE_SIZE)
    printf("Element cannot be inserted\n"); }

void search() {
    int key, index, i, flag = 0, hkey;
    printf("Enter search element\n");
    scanf("%d", &key);
    hkey = key % TABLE_SIZE;
    for (i = 0; i < TABLE_SIZE; i++) {
        index = (hkey + i * i) % TABLE_SIZE;
        if (h[index] == key) {
            printf("value is found at index %d",
                index); break; }
    }
    if (i == TABLE_SIZE)
        printf("value is not found\n"); }

void display() { int i;
    printf("Elements in the hash table are\n");
    for (i = 0; i < TABLE_SIZE; i++)
        printf("At index %d value = %d",
            i, b[i]); }

void main() {
    int opt, i;

```

```

while (1) {
    printf("\n Press 1. Insert 2. Display 3. Search 4. Exit\n");
    scanf("%d", &opt);
    switch (opt) {
        case 1: insert();
                break;
        case 2: display();
                break;
        case 3: search();
                break;
        case 4: exit(0);
    }
}
}
}

```